МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

Кафедра «Интеллектуальные информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине «Математические основы интеллектуальных систем»

за 1 семестр

Тема: «Нахождение компонент связности неориентированного графа»

Вариант 1

Выполнил:

студент 2 курса

группы ИИ29

(2 подгруппа)

Афанасьев Д.Р.

Проверил:

Козинский А.А.

Брест 2026

Цель: Изучить компоненты связанности, методы обходов графов, алгоритм нахождения компонент связности графа.

Ход работы:

1.	9	10	(1,2),(1,4),(1,6),(2,3), 2,5),(3,5), (4,5),(4,6),(7,8),(7,9)

Задание 1: Построить матрицу смежности и инцидентности для заданного графа. Изобразить граф.

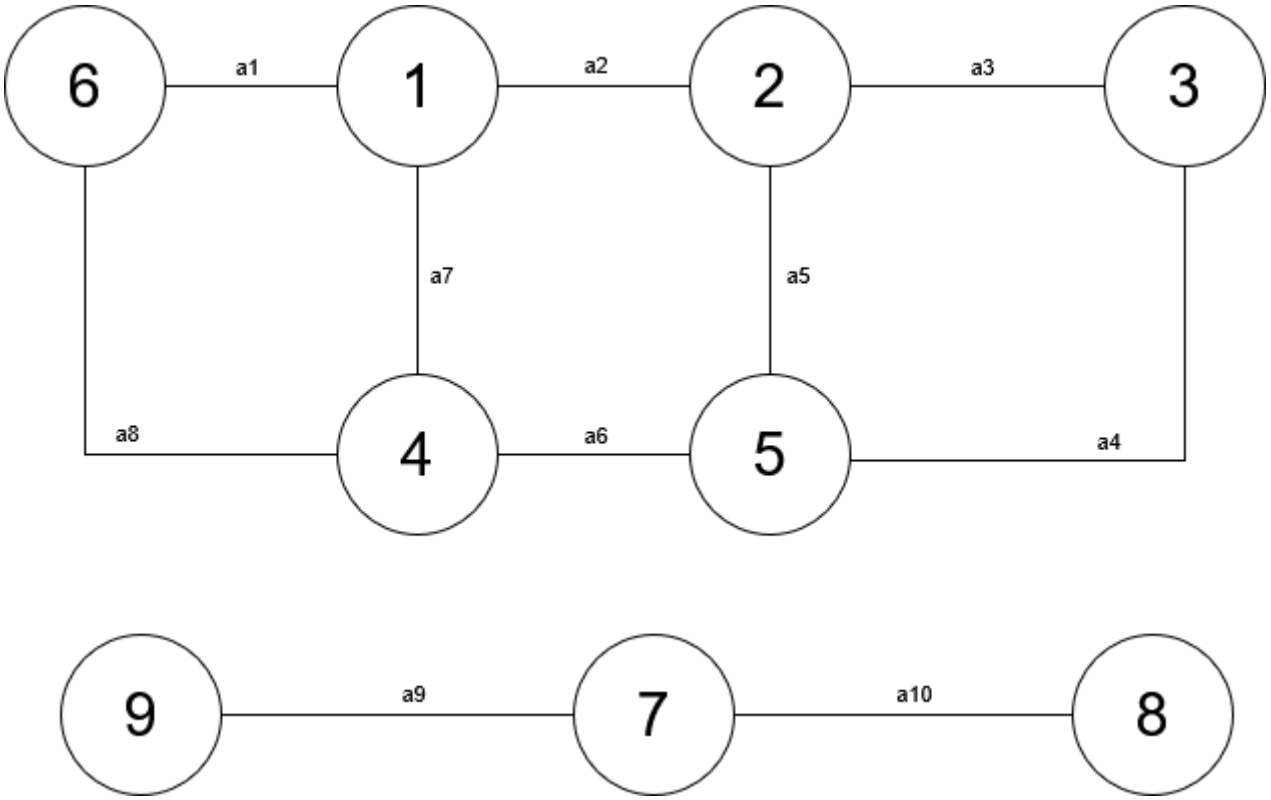
Матрица смежности:

	1	2	3	4	5	6	7	8	9
1	0	1	0	1	0	1	0	0	0
2	1	0	1	0	1	0	0	0	0
3	0	1	0	0	1	0	0	0	0
4	1	0	0	0	1	1	0	0	0
5	0	1	1	1	0	0	0	0	0
6	1	0	0	1	0	0	0	0	0
7	0	0	0	0	0	0	0	1	1
8	0	0	0	0	0	0	1	0	0
9	0	0	0	0	0	0	1	0	0

Матрица инцидентности:

	a1	a2	a3	a4	a5	a6	a7	a8	a9	a10
1	1	1	0	0	0	0	1	0	0	0
2	0	1	1	0	1	0	0	0	0	0
3	0	0	1	1	0	0	0	0	0	0
4	0	0	0	0	0	1	1	1	0	0
5	0	0	0	1	1	1	0	0	0	0
6	1	0	0	0	0	0	0	1	0	0
7	0	0	0	0	0	0	0	0	1	1
8	0	0	0	0	0	0	0	0	0	1
9	0	0	0	0	0	0	0	0	1	0

Граф:



Задание 2: Используя поиск в глубину и поиск в ширину написать программу, определяющую число компонент связности графа. Методы представляются в виде функций.

Код программы:

```
#pragma execution_character_set("utf-8")
#include <stdio.h>
#include <locale.h>

#define MAX_NODES 100

int adj[MAX_NODES][MAX_NODES] = {0};

// Функция добавления ребра в неориентированный граф
void addEdge(int u, int v) {
    if (u >= 1 && u < MAX_NODES && v >= 1 && v < MAX_NODES) {
        adj[u][v] = 1;
        adj[v][u] = 1;
    } else {
        printf("Предупреждение: вершины %d и %d выходят за допустимый диапазон (1..%d)\n", u, v, MAX_NODES - 1);
    }
}

//Поиск в глубину (DFS)
void DFS(int u, int visited[], int n) {
    visited[u] = 1;
    for (int v = 1; v <= n; v++) {
        if (adj[u][v] == 1 && !visited[v]) {
            DFS(v, visited, n);
        }
    }
}

// Подсчет компонент связности с помощью DFS
int countComponentsDFS(int n) {
    int visited[MAX_NODES] = {0};
```

```
    int count = 0;

    for (int i = 1; i <= n; i++) {
        if (!visited[i]) {
            DFS(i, visited, n);
            count++;
        }
    }
    return count;
}

//Поиск в ширину (BFS)
void BFS(int start, int visited[], int n) {
    int queue[MAX_NODES];
    int front = 0, rear = 0;

    visited[start] = 1;
    queue[rear++] = start;

    while (front < rear) {
        int u = queue[front++];

        for (int v = 1; v <= n; v++) {
            if (adj[u][v] == 1 && !visited[v]) {
                visited[v] = 1;
                queue[rear++] = v;
            }
        }
    }
}

// Подсчет компонент связности с помощью BFS
int countComponentsBFS(int n) {
```

```
    int visited[MAX_NODES] = {0};
    int count = 0;

    for (int i = 1; i <= n; i++) {
        if (!visited[i]) {
            BFS(i, visited, n);
            count++;
        }
    }
    return count;
}

int main() {
    setlocale(LC_ALL, "ru_RU.UTF-8");

    int n, m;

    for (int i = 0; i < MAX_NODES; i++) {
        for (int j = 0; j < MAX_NODES; j++) {
            adj[i][j] = 0;
        }
    }

    printf("Введите количество вершин в графе (n, от 1 до %d): ", MAX_NODES -
1);
    if (scanf("%d", &n) != 1 || n < 1 || n >= MAX_NODES) {
        printf("Некорректное количество вершин. Завершение программы.\n");
        return 1;
    }

    printf("Введите количество ребер в графе (m): ");
    if (scanf("%d", &m) != 1 || m < 0) {
        printf("Некорректное количество ребер. Завершение программы.\n");
```

```
        return 1;
    }

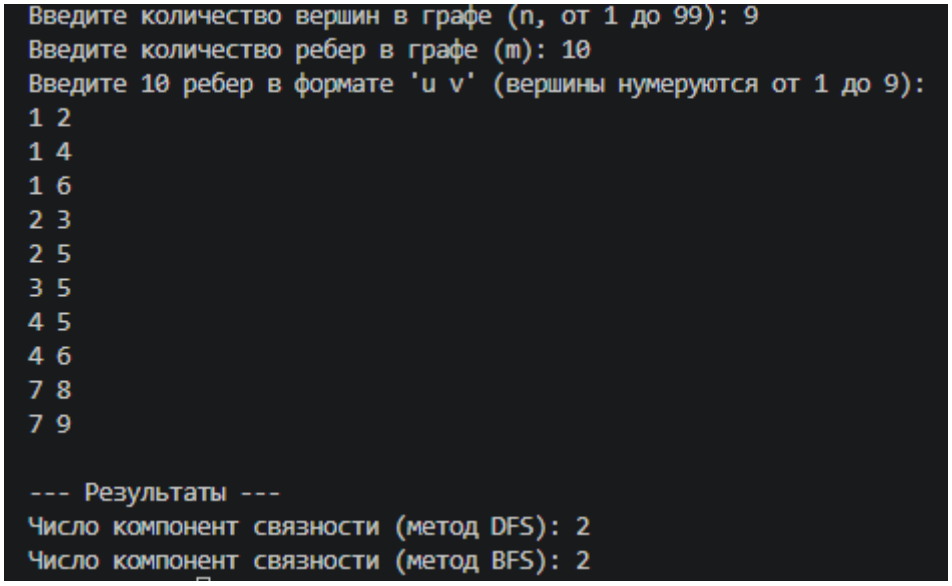
    printf("Введите %d ребер в формате 'u v' (вершины нумеруются от 1 до %d):\n", m, n);
    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        addEdge(u, v);
    }

    int dfs_count = countComponentsDFS(n);
    int bfs_count = countComponentsBFS(n);

    printf("\n--- Результаты ---\n");
    printf("Число компонент связности (метод DFS): %d\n", dfs_count);
    printf("Число компонент связности (метод BFS): %d\n", bfs_count);

    return 0;
}
```

Тест 1:



Тест 2:

```
Введите количество вершин в графе (n, от 1 до 99): 6
Введите количество ребер в графе (m): 8
Введите 8 ребер в формате 'u v' (вершины нумеруются от 1 до 6):
1 2
1 5
1 6
2 3
3 4
4 5
5 6

--- Результаты ---
Число компонент связности (метод DFS): 1
Число компонент связности (метод BFS): 1
```

Тест 3:

```
Введите количество вершин в графе (n, от 1 до 99): 8
Введите количество ребер в графе (m): 6
Введите 6 ребер в формате 'u v' (вершины нумеруются от 1 до 8):
1 6
1 8
2 5
3 6
3 8
4 5

--- Результаты ---
Число компонент связности (метод DFS): 3
Число компонент связности (метод BFS): 3
```

Вывод: в ходе лабораторной работы я изучил компоненты связанности, методы обходов графов, алгоритм нахождения компонент связности графа.